

DSA Master Handbook - Volume 1

Detailed interview preparation guide with statements, examples, patterns and complexities.

1. Two Sum

Problem Statement: Given an array of integers and a target value, return the indices of two numbers whose sum equals the target.

Example: Input: nums=[2,7,11,15], target=9 Output: [0,1]

Expected Algorithm: Use a hash map to store visited values and their indices. For each element, check whether target-current exists in the map.

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Pattern: Hashing

Interview Notes: Explain brute force first, then optimize. Be able to derive complexity and discuss edge cases.

2. Best Time to Buy and Sell Stock

Problem Statement: Find the maximum profit obtainable from one buy and one sell operation.

Example: Input: [7,1,5,3,6,4] Output: 5

Expected Algorithm: Track minimum price seen so far and update maximum profit while traversing.

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Pattern: Sliding Window / Greedy

Interview Notes: Explain brute force first, then optimize. Be able to derive complexity and discuss edge cases.

3. Longest Substring Without Repeating Characters

Problem Statement: Return the length of the longest substring without repeating characters.

Example: Input: abcabcbb Output: 3

Expected Algorithm: Maintain a sliding window and a set/hash map for character positions.

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Pattern: Sliding Window

Interview Notes: Explain brute force first, then optimize. Be able to derive complexity and discuss edge cases.

4. Valid Parentheses

Problem Statement: Determine whether brackets are balanced.

Example: Input: ()[]{} Output: True

Expected Algorithm: Push opening brackets into stack and match closing brackets.

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Pattern: Stack

Interview Notes: Explain brute force first, then optimize. Be able to derive complexity and discuss edge cases.

5. Reverse Linked List

Problem Statement: Reverse a singly linked list.

Example: 1->2->3 becomes 3->2->1

Expected Algorithm: Use three pointers: prev, curr, next.

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Pattern: Linked List

Interview Notes: Explain brute force first, then optimize. Be able to derive complexity and discuss edge cases.

6. Binary Search

Problem Statement: Search a target in a sorted array.

Example: [1,3,5,7,9], target=7 => index 3

Expected Algorithm: Repeatedly divide search space in half.

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

Pattern: Binary Search

Interview Notes: Explain brute force first, then optimize. Be able to derive complexity and discuss edge cases.

7. Number of Islands

Problem Statement: Count connected components of land in a 2D grid.

Example: Grid of 1s and 0s -> count islands

Expected Algorithm: Run DFS/BFS from every unvisited land cell.

Time Complexity: $O(m*n)$

Space Complexity: $O(m*n)$

Pattern: Graph Traversal

Interview Notes: Explain brute force first, then optimize. Be able to derive complexity and discuss edge cases.

8. House Robber

Problem Statement: Maximize money stolen without robbing adjacent houses.

Example: [2,7,9,3,1] => 12

Expected Algorithm: Dynamic Programming: $dp[i] = \max(dp[i-1], dp[i-2] + \text{nums}[i])$

Time Complexity: $O(n)$

Space Complexity: $O(n)$ or $O(1)$

Pattern: Dynamic Programming

Interview Notes: Explain brute force first, then optimize. Be able to derive complexity and discuss edge cases.

9. Kth Largest Element

Problem Statement: Find kth largest element in an unsorted array.

Example: $[3, 2, 1, 5, 6, 4]$, $k=2 \Rightarrow 5$

Expected Algorithm: Use min heap of size k or Quick Select.

Time Complexity: $O(n \log k)$

Space Complexity: $O(k)$

Pattern: Heap

Interview Notes: Explain brute force first, then optimize. Be able to derive complexity and discuss edge cases.

10. Merge Intervals

Problem Statement: Merge overlapping intervals.

Example: $[[1, 3], [2, 6], [8, 10]] \Rightarrow [[1, 6], [8, 10]]$

Expected Algorithm: Sort intervals and merge overlapping ranges.

Time Complexity: $O(n \log n)$

Space Complexity: $O(n)$

Pattern: Greedy

Interview Notes: Explain brute force first, then optimize. Be able to derive complexity and discuss edge cases.

Recommended 200+ Question Roadmap

Arrays (25), Strings (20), Two Pointers (15), Sliding Window (15), Stack (15), Queue (10), Linked List (20), Binary Search (15), Trees (35), Heaps (15), Graphs (30), Backtracking (15), Dynamic Programming (40), Greedy (15), Tries, Segment Trees, DSU, LRU Cache, LFU Cache and Advanced Algorithms.